

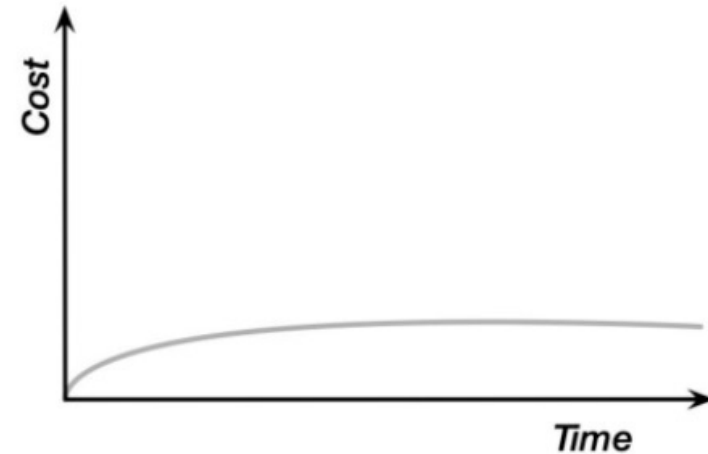
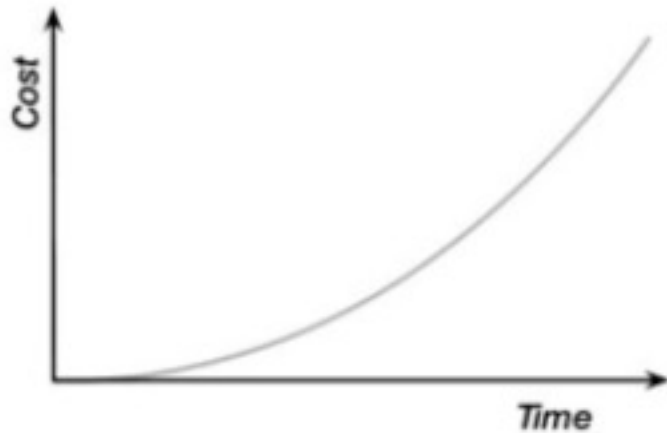
eXtreme Programming

Comprendre XP permettra de comprendre Agile

01 Le concept clé

Le point clé; le *pourquoi?* de XP

- Les valeurs et pratiques XP ont pour objectif de changer l'hypothèse du coût du changement



Comment faire ?

Question :

A votre avis, comment peut on essayer d'arriver à minimiser le coût de changement

- Peut-on refactorer le logiciel sans créer de bug ou d'effet de bord ?
- Peut on tester de manière continue le logiciel (automatisation des tests) ?
- Peut-on livrer rapidement les premiers incréments au client ?
- Tout le monde a-t-il la même vision du logiciel ?
- ...

02 Le framework XP

Valeurs, principes et pratiques

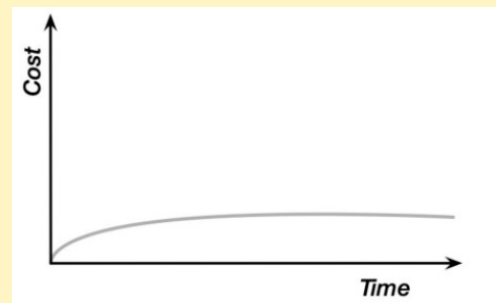
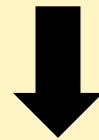
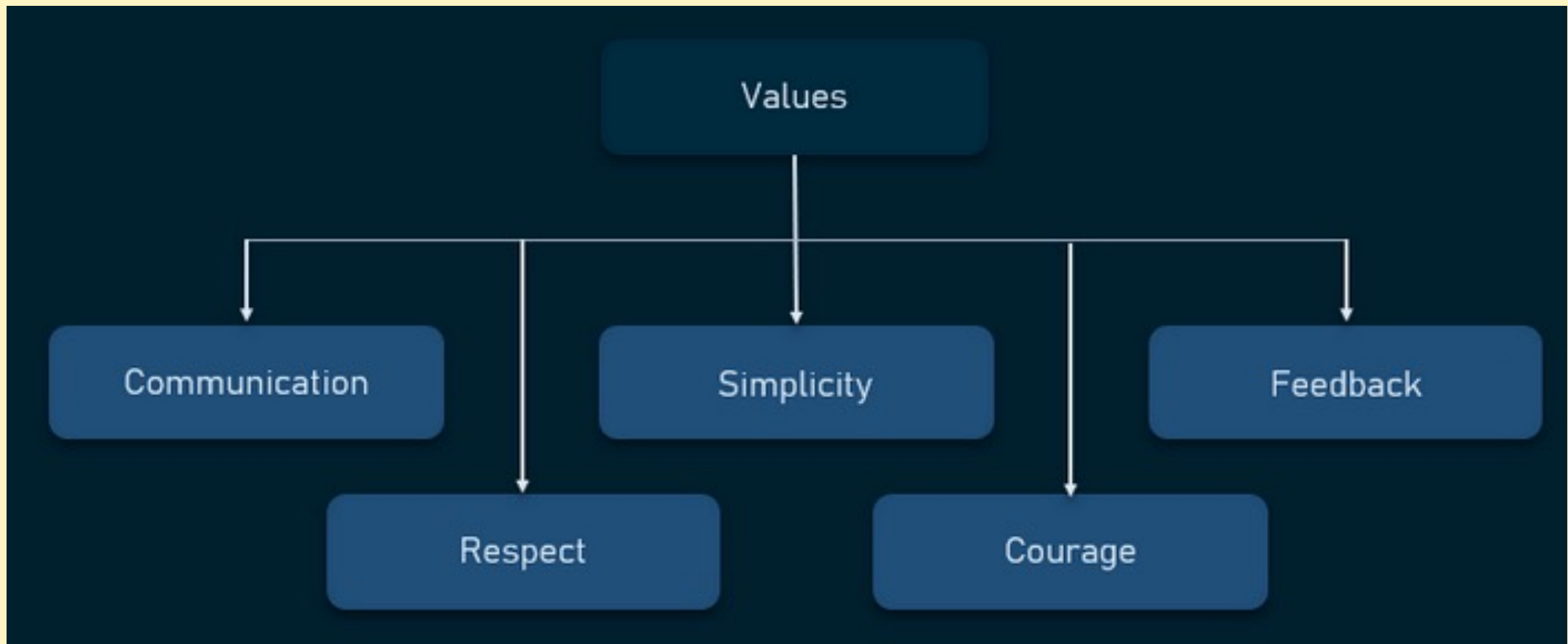
01 Les 5 valeurs XP

Valeurs → Principes → Pratiques

Prenons un exemple très simple hors développement.

- Valeur : « **la sécurité** ».
- De cette valeur découle un principe : « **réduire les risques avant qu'un accident arrive** ».
- Et de ce principe découlent des pratiques concrètes : **mettre une ceinture, respecter les limitations de vitesse, entretenir les freins.**

Une valeur sert principalement à **orienter les décisions** lorsqu'aucune règle précise ne te dit quoi faire.



02 Les principes XP

These principles will help us as we choose between alternatives. We will prefer an alternative that meets the principles more fully to one that doesn't. Each principle embodies the values. A value may be vague. One person's simple is another person's complex. A principle is more concrete. Either you have rapid feedback or you don't.

Extreme Programming Explained (First Edition) – Chapitre 8

Kent Beck

Principes fondamentaux

<https://martinfowler.com/bliki/PrinciplesOfXP.html> /

- Rapid Feedback
- Assume Simplicity
- Incremental Change
- Embracing Change
- Quality Work

Principes secondaires

<https://martinfowler.com/bliki/PrinciplesOfXP.html> /

- Teach Learning
- Small Initial Investment
- Play to Win
- Concrete Experiments
- Open, honest Communication
- Work with people's instincts - not against them
- Accepted Responsibility
- Local Adaptation
- Travel Light
- Honest Measurement

03 Pratiques XP

Pratiques primaires (et secondaires)

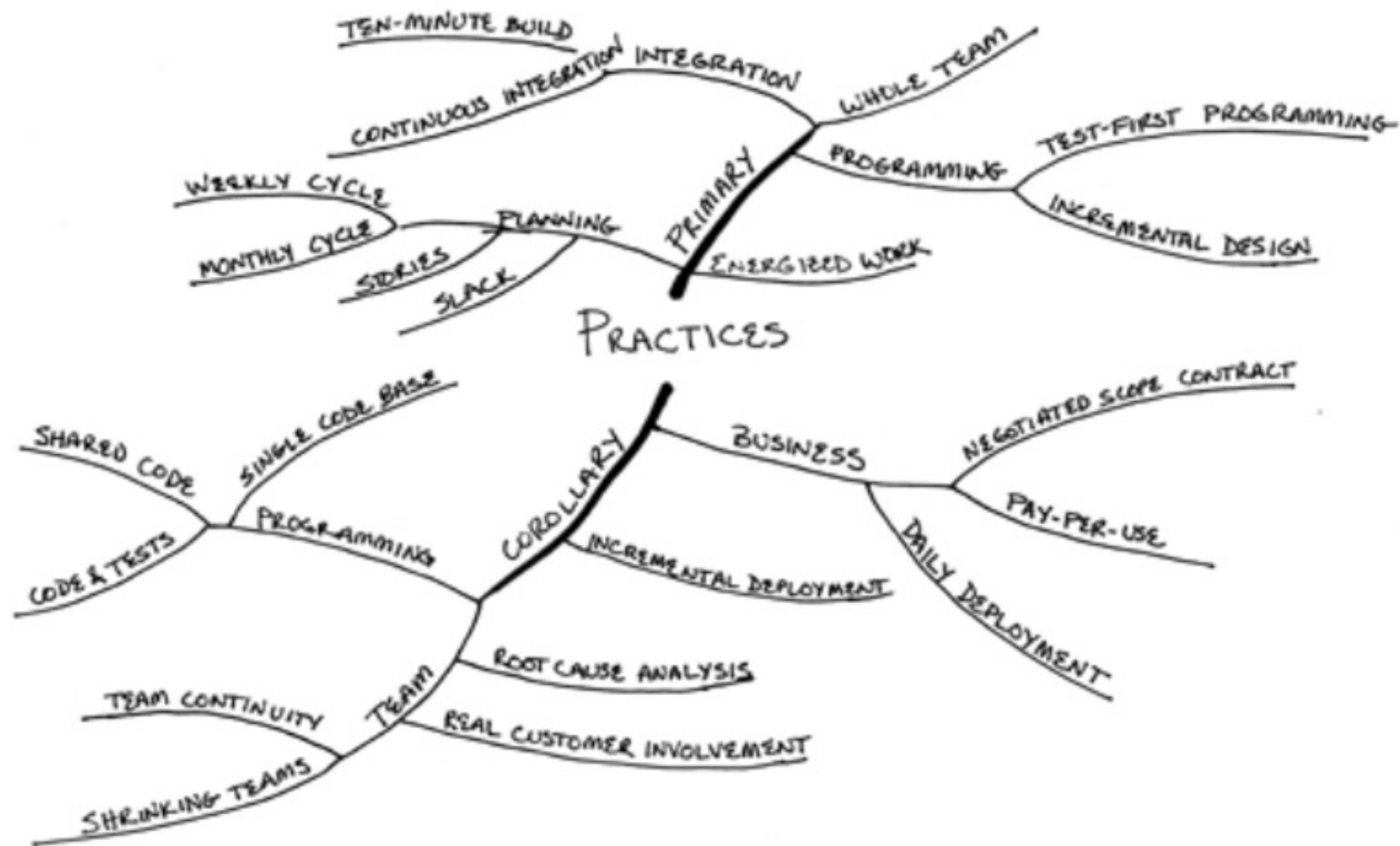


Figure 3. Summary of practices

The customer role

The people in the Customer role choose what will deliver business value, choose what to do first and what to defer, and define the tests to show that the system does what it needs to.

Every software project needs to deliver business value. To be successful, the team needs to build the right things, in the right order, and to be sure that what they build actually works. Of course this can't be done without programmers, but in fact the customer's role is critical in steering that process to success.

The customer role on an XP project can be filled by one person, or by several. The team will be most effective if the customer is on-site, present with the team, full time. We'll discuss some details in *On-site Customer* (page 31). Here, we'll talk in more general terms about what the customer does. If you're the XP customer, we're talking to you.

Note that we say "the customer", not "the customers". Whether they are one or many people, the XP customer always speaks with one voice. The determination of what will have business value, and the order of building that value, rests solely with the customer. (Don't worry, you get lots of help and advice. But ultimately, you get to make the call.)

An XP team plans and builds software in terms of "stories". Stories are just that — individual stories about how the system needs to work. Each story describes one thing that the system needs to do. Each story must be understood well enough that the programmers can estimate its difficulty. And each story must be testable.

As the customer, you express what must be done in terms of stories. For a project spanning a few months, there may be 50 or 100 stories. Larger projects of course have more stories. We'll talk more about the details in *User Stories* (page 37).

You probably have a delivery date in mind, though some projects have a fixed feature list rather than a fixed date. We are not content to imagine that everything that you can think of will be done by a given date. Neither should you be. Instead, the XP process lets the team predict, more and more accurately, how much work can be done in any given time period. Using this information, you manage project scope — choosing what to do now and what to defer until later — to ensure successful delivery.

You, the customer, have the critical responsibility to choose the stories that will provide the most valuable features, the highest business value, by the desired delivery date. The XP development process lets you

choose among the stories with great flexibility. There's not much limitation on what can be done first and what second. This is by design: if you are to choose the stories for successful on-time release, you must have the flexibility to make the choice as independently as possible. Read more about this process in *Customer Defines Release* (page 71) and *Iteration Planning* (page 79).

Finally, you specify tests that show whether the stories have been correctly implemented. These *Acceptance Tests* (page 45), whether built by the programmers, by an independent tester, or by you yourselves, provide confidence that the system really does what it needs to do.

Define business value, decide what to do and what to defer, and define the tests to show that the system works. These are your key responsibilities as the XP customer.

Extreme programming installed – chap1

- **On-site Customer** : le client fait partie intégrante de l'équipe et guide le développement en fonction des besoins métier.
- **User Stories** : les besoins sont découpés en petites histoires décrivant chacune une fonctionnalité compréhensible, estimable et testable.
- **Customer Defines Release** : le client sélectionne les stories qui doivent composer une livraison afin de maximiser la valeur apportée à la date souhaitée.
- **Iteration Planning** : à chaque itération, les stories à développer sont choisies en tenant compte des priorités du client et des estimations de l'équipe.
- **Acceptance Tests** : le client définit des tests d'acceptation permettant de vérifier objectivement qu'une story répond bien au besoin attendu.

=> le client décide **quoi construire, dans quel ordre et comment vérifier que cela répond au besoin**

The programmer role

The people in the Programmer role analyze, design, test, program, and integrate the system. The programmers estimate the difficulty of all stories, and track the pace at which they can deliver stories to the customer.

If the project is to deliver business value, each story must be understood. Software must be designed, tested, and built to implement that story, and all the software must be brought together into a coherent whole. That is the job of those in XP's programmer role. If you're a programmer on an XP project, we're talking to you.

In Extreme Programming, the emphasis is on programming. Everything we do looks like programming and is focused on the most critical artifact of software development, the program.

Build the system in small releases, so that the customer benefit is maximized and you get the best possible **feedback** on how you're doing. We talk about this in *Small Releases* (page 65), *Customer Defines Release* (page 71), and *Iteration Planning* (page 79).

Base the program on simple, clear design. This lets you produce quality software quickly. There's more discussion of this in *Code Quality* (page 103), and *A True Story* (page 257). As you learn more about what the design wants to be, improve the design using *Refactoring* (page 95).

XP isn't slash and burn programming, not code and fix, not at all. Extreme Programming is about careful and continuous design, rapid feedback from extensive testing, and the maintenance of relentlessly clear and high-quality code.

Keep the system integrated at all times, so there's always a good version to look at. Keeping integrated lets you go rapidly without stepping on each others' toes. See *Continuous Integration* (page 96).

Share the ownership of all the code, so no one has to wait and everyone feels able to make everything better. See *Collective Code Ownership* (page 93), and *Releasing Changes* (page 145). Share a single *Coding Standard* (page 97) as well, whether you evolve your own or adopt one from elsewhere. Make everyone's code look alike—it helps with communication and team focus. Express your individuality in the way you wear your XP ball cap.

Make sure that the system always works, using comprehensive unit tests of your own making, as well as the customer's acceptance tests. These tests allow rapid change, and support collective code ownership by keeping change from introducing mistakes. See *Unit Tests* (page 113), *Acceptance Tests* (page 45), *Everything that could possibly break* (page 265), and *Test-first, by Intention* (page 129).

Write all production code in pairs, for maximum speed and cross-training, in support of shared code ownership and rapid progress, as described in *Pair Programming* (page 107).

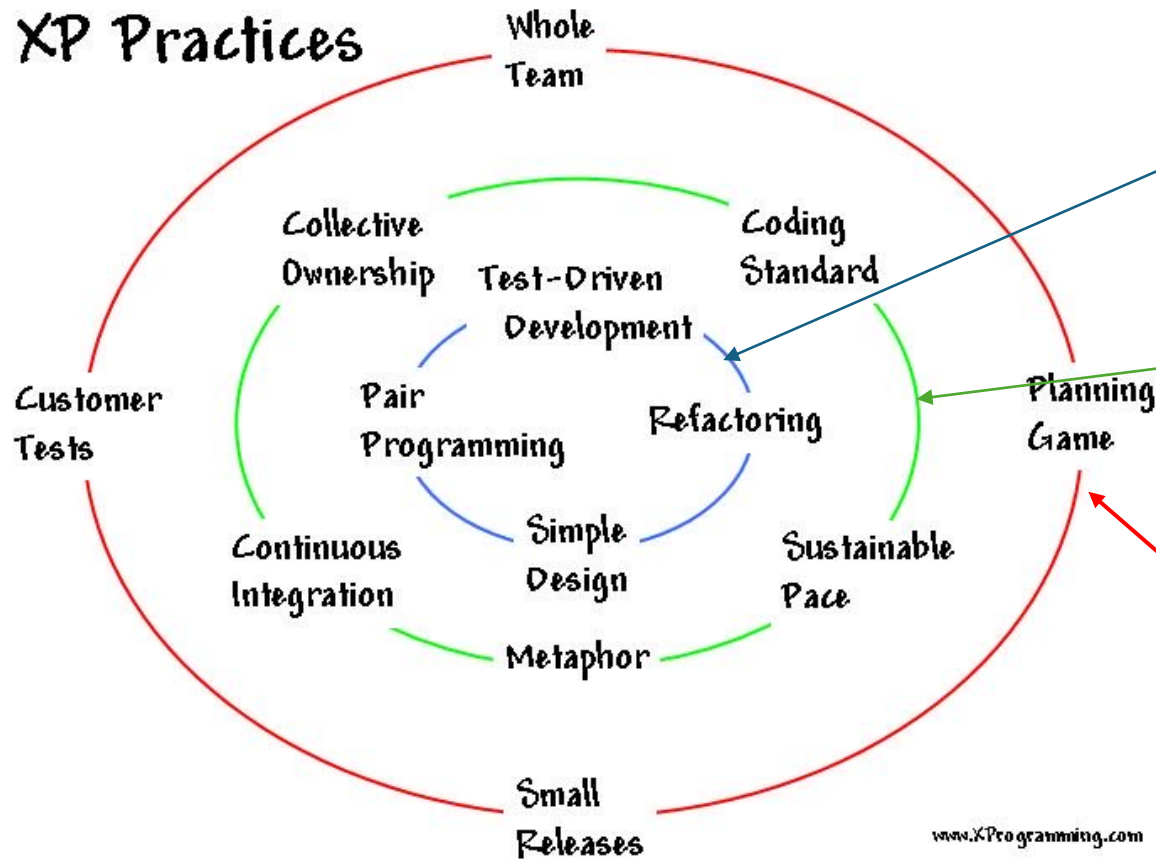
Extreme Programming is an approach to software development that lets programmers do what they do best — program — while giving the customers what they need most — business value. It's a win-win approach and fun too.

Extreme programming installed – chap1

=> les programmeurs **estiment, conçoivent, développent, testent et intègrent continuellement** le logiciel tout en maintenant un code simple, partagé et de haute qualité.

- **Small Releases** : construire et livrer le système par petites versions afin d'apporter rapidement de la valeur et d'obtenir régulièrement du feedback.
- **Refactoring** : améliorer continuellement la structure du code sans modifier son comportement afin de conserver un design propre.
- **Continuous Integration** : intégrer fréquemment les changements afin de disposer en permanence d'une version fonctionnelle et d'éviter les conflits.
- **Collective Code Ownership** : tout développeur peut modifier n'importe quelle partie du code afin de supprimer les dépendances individuelles et favoriser l'amélioration collective.
- **Coding Standard** : utiliser des conventions de code communes pour rendre le code homogène, lisible et facilement compréhensible par toute l'équipe.
- **Unit Tests** : maintenir une suite complète de tests unitaires pour vérifier continuellement que le système fonctionne correctement.
- **Test-First** : écrire les tests avant le code de production afin de guider l'implémentation et de vérifier immédiatement son comportement.
- **Pair Programming** : écrire le code de production à deux développeurs afin d'améliorer la qualité, le partage de connaissances et la propriété collective du code.

XP Practices



Orienté développeurs
(Coding Circle)

- day-to-day pratiques
- Chaque feature dans un design simple, codé en PairProgramming en utilisant le TDD

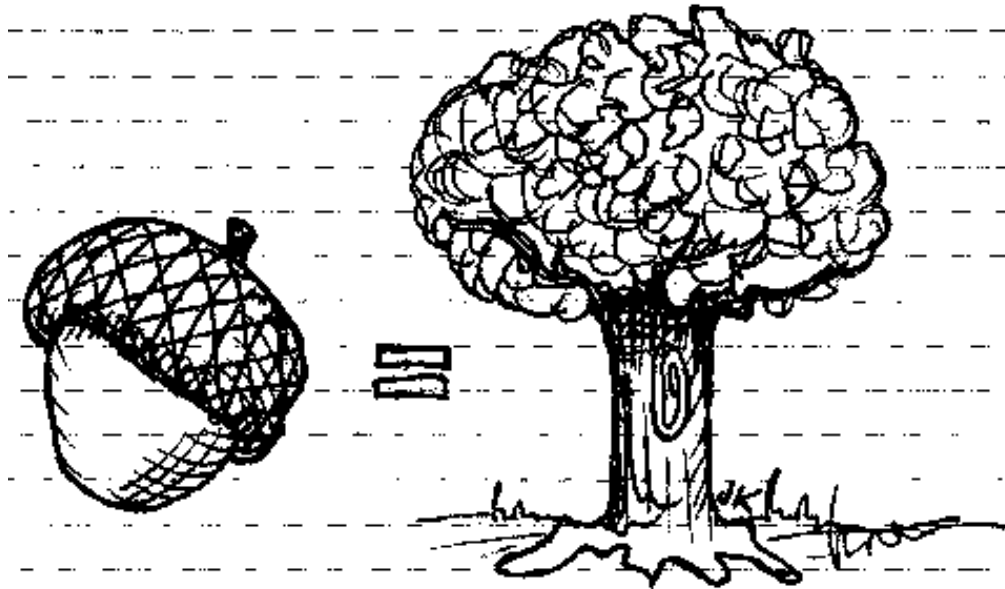
Orienté équipes
(Team Circle)

- Le logiciel est codé pour permettre de la CI,
- Les personnes ont une vue globale et une connaissance partagée

Orienté projet/produit
(Project Circle)

Small Release

The outermost XP cycle is the release. Small and frequent releases provide early benefit to the customer while providing **early feedback** to the programmers. Here are some thoughts on how to make it happen.



Iteration Planning

Inside each release, an Extreme team plans just a few weeks at a time, with clear objectives and solid estimates.

La planification d'itération comporte trois étapes essentielles :

1. Le client présente les « user stories » ;
2. L'équipe réfléchit ensemble aux tâches techniques ;
3. Les programmeurs se répartissent les tâches et établissent des estimations.

04 Circle of life

Très simple mais souvent oublié

An XP project succeeds when the customers select business value to be implemented, based on the team's measured ability to deliver functionality over time.

Extreme Programming Installed (chapitre 2)

Ron Jeffries

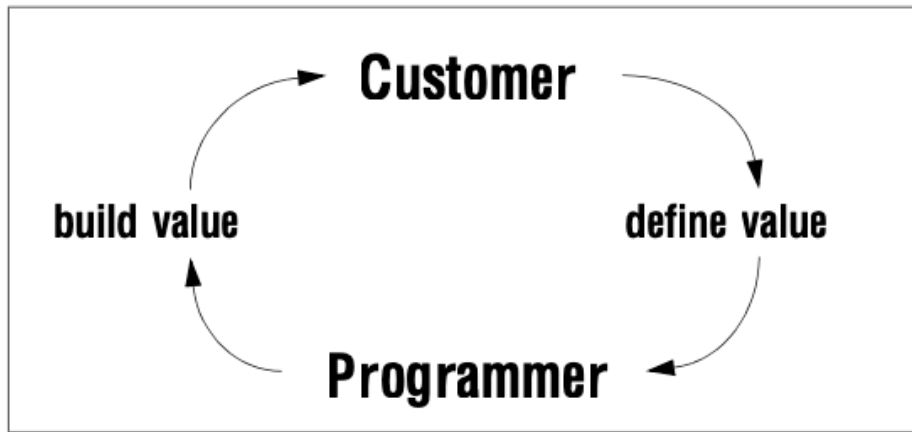


Figure 2.1 Customer defines, programmer builds.

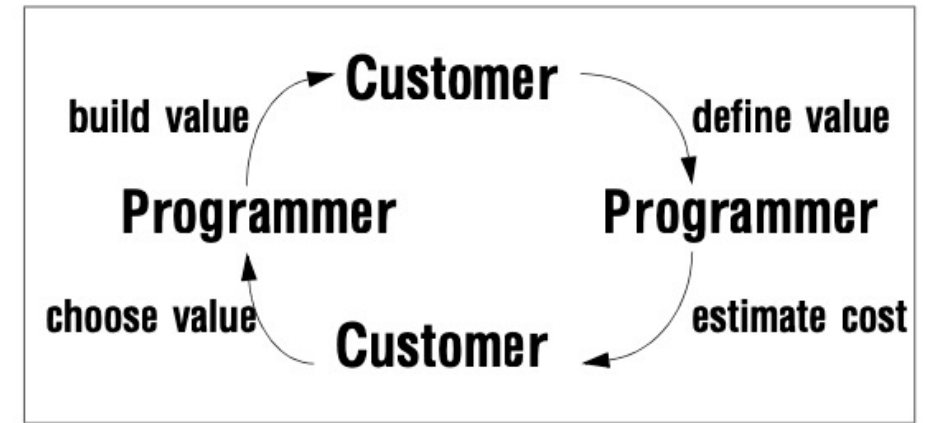


Figure 2.2 Programmer estimates, customer chooses.

Question :

Mais cela ne suffit pas, que manque-t-il ?

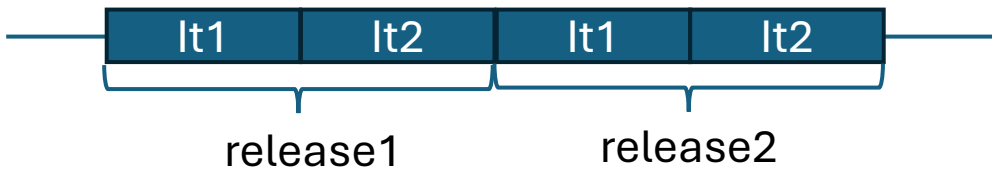
Pour décider quoi développer et à quel moment, **le client doit donc connaître le coût des fonctionnalités** qu'il demande. Les développeurs lui fournissent ces estimations en s'appuyant sur leur expérience et leur connaissance technique.

If what you want to do is manage your project to the most successful conclusion,
then follow the circle of life: **define, estimate, choose, build.**

Extreme Programming Installed (chapitre 2 - conclusion)

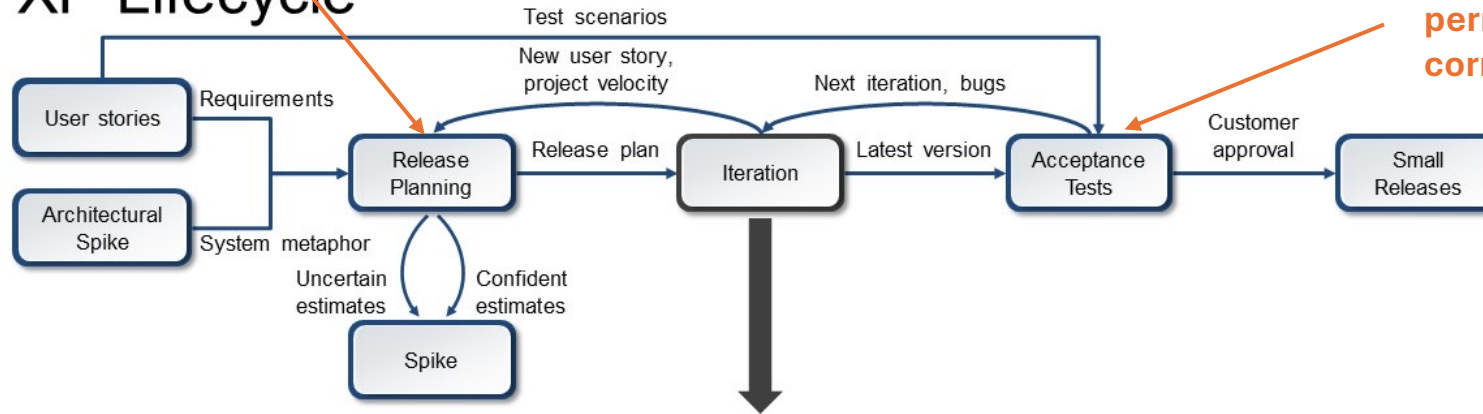
Ron Jeffries

Cycle XP

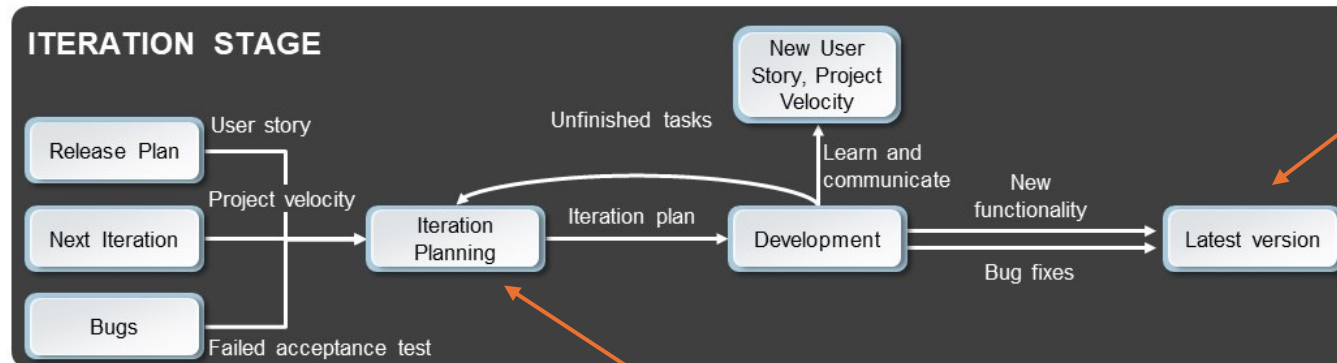


1. Le *Release Planning* sert à déterminer les **User Stories visées pour une release**

XP Lifecycle



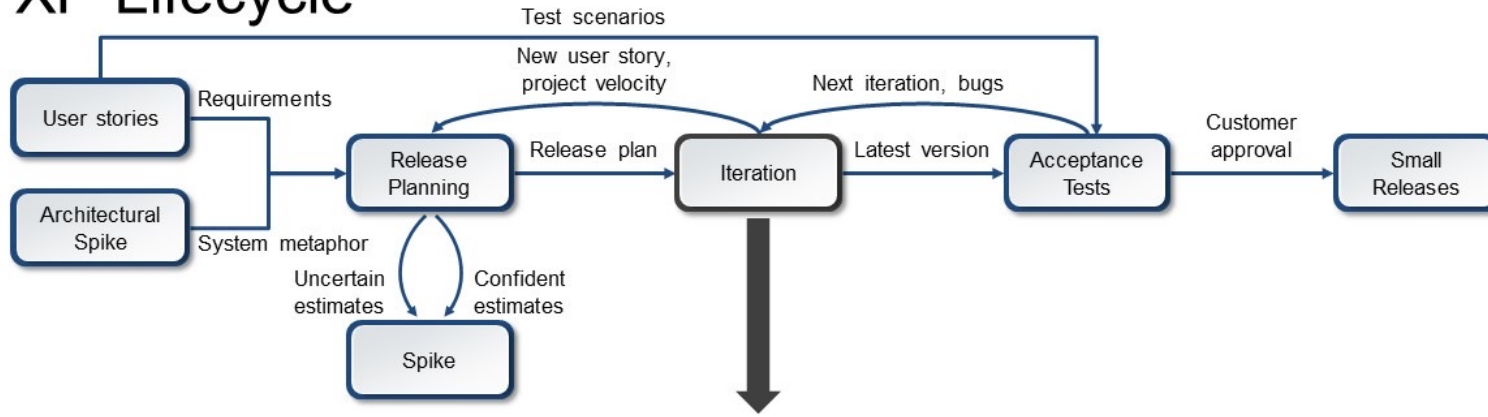
4. Dans l'US, le client à préciser les **critères permettant de vérifier que la story est correctement réalisée.**



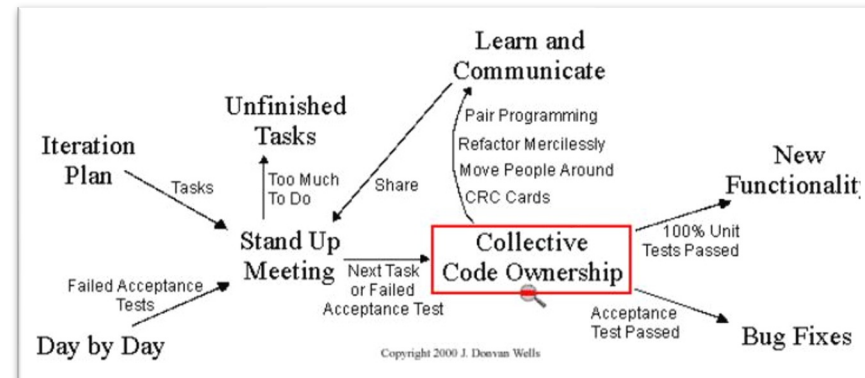
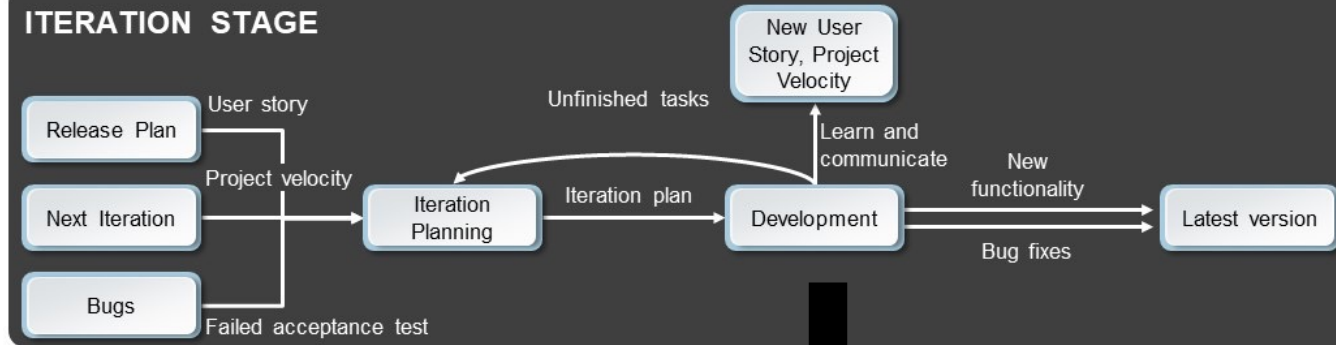
3. À la fin d'une itération, on obtient un **incrément logiciel fonctionnel.**

2. Chaque itération possède son propre *Iteration Planning*, réalisé **au début de cette itération.**

XP Lifecycle

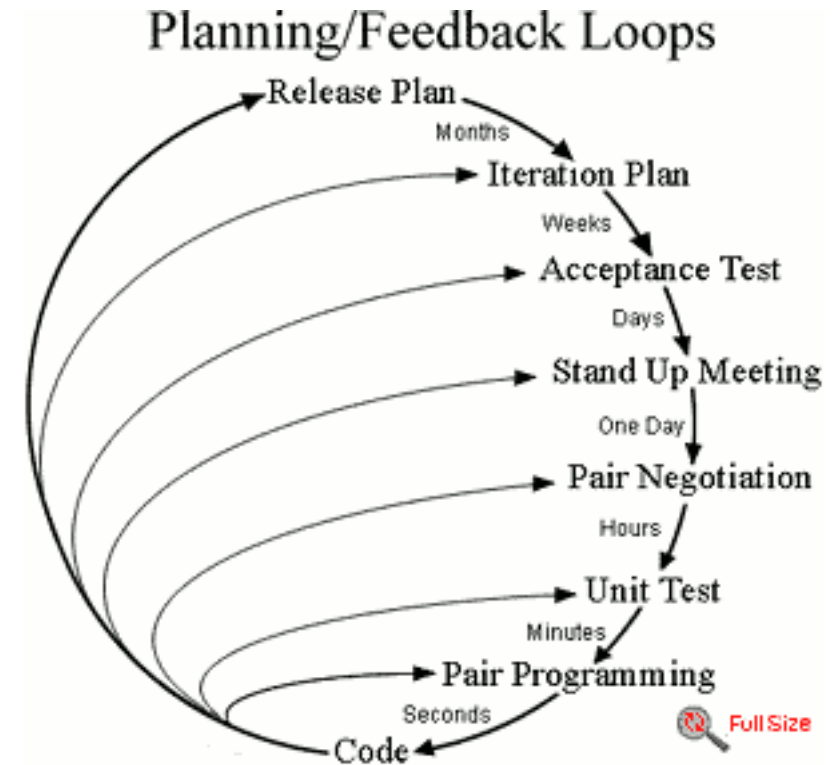


ITERATION STAGE



Obtenir du feedback

- À l'échelle d'une release, il peut s'écouler plusieurs semaines ou mois avant d'observer le résultat auprès du client.
- À l'échelle d'une itération, le feedback est ramené à quelques jours ou quelques semaines
- À l'échelle d'un test unitaire, le feedback est presque immédiat pour savoir si une modification fonctionne ou introduit une régression.



Résumé

- 1 XP se concentre sur des pratiques techniques: continuous testing, code reviews, TDD, pair-programming
- 2 Met l'accent sur des itérations rapides afin de collecter du feedback
- 3 XP explique « comment » faire du développement logiciel (Scrum ne dit rien). De nombreuses équipes Scrum adoptent donc des valeurs, principes et pratiques XP